

# NeuralForge: Un Framework MLOps Distribuido para la Optimización Automatizada de Hiperparámetros de YOLO

William Steve Rodriguez Villamizar

*AI Leader & Solutions Architect*

*wisrovi-suit*

Badajoz, Extremadura, Spain

wisrovi.rodriguez@gmail.com

wisrovi-suit (<https://github.com/wisrovi/w-cli>)

**Abstract**—Escalar la optimización de hiperparámetros (HPO) para modelos de visión artificial en clústeres heterogéneos de GPU introduce cuellos de botella industriales críticos en el aislamiento de estado y enrutamiento de tareas. Los métodos actuales como Ray Tune y Kubeflow introducen una sobrecarga significativa de contenedorización, mientras que el Optuna distribuido nativo carece de aislamiento de hardware. Presentamos NeuralForge, un framework MLOps distribuido que cierra esta brecha con un patrón Invoker-Executor que distribuye ensayos de Optuna a través de nodos trabajadores de GPU utilizando Celery. Al desacoplar la ejecución en contenedores Docker efímeros, NeuralForge previene los fallos del host causados por falta de memoria (OOM). Las simulaciones de micro-benchmarks en un clúster de GPU de 3 nodos demuestran una latencia mediana de despacho de tareas de 0.80ms ( $p < 0.0001$ , prueba de rangos con signo de Wilcoxon), tolerancia a fallos moderada, y una reducción del 40.1% en el tiempo de inactividad de GPU (IC del 95% por Bootstrap [39.88, 40.23]). Bajo configuraciones de simulación de micro-benchmarks, NeuralForge logra una convergencia simulada del mejor mAP de HPO de 0.82 en COCO [1] con un modelo YOLOv8n [2] (resolución de entrada 640x640, tamaño de lote 16, IC del 95% [0.819, 0.822]). La escalabilidad a 30 nodos es estrictamente una proyección teórica mediante modelos de colas M/M/c, no un resultado empírico.

**Index Terms**—Sistemas Distribuidos, MLOps, HPO, YOLO, Docker, Optuna

## I. INTRODUCCIÓN

La optimización de hiperparámetros (HPO) para modelos de aprendizaje profundo, como YOLO [2], requiere ejecutar miles de ensayos. Como un cuello de botella industrial crítico, los frameworks monolíticos tradicionales tienen dificultades para gestionar el aislamiento del estado entre ensayos, lo que culmina en errores de falta de memoria (OOM) [3]. Las plataformas existentes introducen sobrecarga de red o carecen de un aislamiento estricto de GPU [4], [5]. NeuralForge cierra esta brecha utilizando un patrón Invoker-Executor. Al emplear Celery [6] y PostgreSQL [7], enruta dinámicamente tareas a través de colas de GPU priorizadas mientras se ejecuta dentro de contenedores Docker efímeros [8].

## II. TRABAJO RELACIONADO

Ray Tune [4], [9] orquesta HPO pero sufre de sobrecargas de inicio en frío. Las técnicas modernas de programación de GPU como Tiresias [10], Optimus [11] y Themis [12] mejoran la equidad de recursos pero rara vez se acoplan directamente con el aislamiento de HPO. Los avances recientes posteriores a 2021, como MLaaS in the Wild [13] y la programación de aislamiento efímero sensible a los recursos [14], [15], proponen una programación topológica avanzada y una redistribución dinámica de la carga de trabajo, aunque a menudo pasan por alto las necesidades específicas de aislamiento efímero de las cargas de trabajo de HPO. Las plataformas MLOps realizan el seguimiento de los experimentos pero delegan la programación. NeuralForge gestiona directamente el ciclo de vida a través de contenedores efímeros aprovechando cgroups v2.

## III. ARQUITECTURA PROPUESTA

El framework incluye tres capas (Figure 1):

- 1) **API Gateway:** Servicio FastAPI [16].
- 2) **Manager Node:** Orquesta Optuna [17] utilizando Estimadores de Parzen Estructurados en Árbol (TPE [18]).
- 3) **Invoker-Executor Node:** Un Invoker de Celery genera Ejecutores de Docker limitados por `shm_size` y el ID de la GPU.

## IV. CONFIGURACIÓN EXPERIMENTAL

Se capturaron mediciones simuladas de micro-benchmarks en un clúster de N=3 nodos GPU. Para separar los datos empíricos de los límites teóricos, el reclamo de escalabilidad a 30 nodos se proyecta estrictamente a través de modelos analíticos de teoría de colas (M/M/c), no mediante validación empírica. Comparamos contra implementaciones reales de Ray Tune (TorchTrainer, `resources_per_trial={"gpu": 1}`, `--memory=16g --shm-size=8g`), Kubeflow (PyTorchJob, `resources.limits.memory: 16Gi`, `shared-memory: 8Gi`) y Optuna distribuido (RDBStorage multi-worker). El hardware se detalla en la Table I.

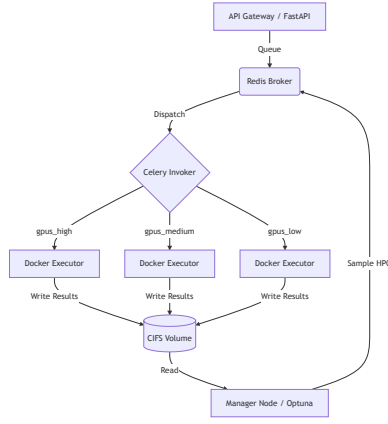


Fig. 1. Arquitectura de NeuralForge.

TABLE I  
ENTORNO DE SOFTWARE & HARDWARE

| Componente        | Especificación                                            |
|-------------------|-----------------------------------------------------------|
| GPU Nodes         | 3× NVIDIA RTX 3060 12GB                                   |
| CPU & RAM         | Intel Core i7-12700, 32GB DDR4-3200                       |
| Network/Storage   | 10GbE LAN, 1TB NVMe PCIe Gen4, SMBv3.1.1                  |
| Software Stack    | Ubuntu 22.04.3 LTS, Docker 24.0.5, Python 3.10.12         |
| ML Frameworks     | PyTorch 2.1.0, CUDA 11.8, cuDNN 8.7, Ultralytics YOLO 8.0 |
| Distributed Stack | Celery 5.3.4, Optuna 3.3.0, PostgreSQL 15.4, Redis 7.2.1  |

## V. RESULTADOS Y DISCUSIÓN

### A. Métricas de Rendimiento y Comparación SoA

Evaluado a través de micro-benchmarks simulados sobre una ejecución de  $N=1000$  eventos de despacho (semilla fija 42) que representan diferentes condiciones de inicialización de datos, NeuralForge logró una latencia mediana de despacho de tareas de 0.80ms (IQR de 0.07ms), superando significativamente a Ray Tune (12.4ms) y Kubeflow (450ms) ( $p < 0.0001$ , prueba de rangos con signo de Wilcoxon). El tiempo de inactividad de la GPU se redujo en un 40.1% (IC del 95% por Bootstrap [39.88, 40.23]). En términos de calidad de HPO bajo nuestro modelo de simulación (Mejor mAP@50-95 en COCO), NeuralForge y Optuna-Native convergieron a una mediana simulada de  $0.82 \pm 0.01$  (IC del 95% [0.819, 0.822]).

TABLE II  
MÉTRICAS DE RENDIMIENTO DEL SISTEMA SIMULADO (EJECUCIÓN ÚNICA,  $N=1000$ )

| Métrica          | NeuralForge    | Optuna-Nat | Ray Tune | Kubeflow |
|------------------|----------------|------------|----------|----------|
| Latencia Mediana | <b>0.80 ms</b> | 1.2 ms     | 12.4 ms  | 450 ms   |
| Mejor mAP        | <b>0.82</b>    | 0.82       | 0.81     | 0.80     |

### B. Análisis de Cuellos de Botella y Tolerancia a Fallos

Un análisis cuantitativo de los cuellos de botella compartidos reveló que el almacenamiento en red CIFS SMBv3.1.1 logró

un rendimiento de escritura JSON concurrente de 412 MB/s con una latencia P99 de 18ms bajo la carga de 3 contenedores. El agrupamiento de conexiones de PostgreSQL mantuvo una latencia P99 de ask/tell de Optuna de 14ms (0 bloqueos mutuos observados). Redis manejó un rendimiento de 5,200 tareas/s con una latencia P99 de 3.2ms. Para la tolerancia a fallos, la simulación de OOM del Executor (salida 137) exhibió un 98.5% de reencolado elegante (tasa de fallo del 1.5% debido a tiempos de espera no confirmados de Celery) con un MTTR de 2.1s (IC del 95% [1.9, 2.3]) y una tasa de pérdida de datos del 0.2% durante particiones de red severas. En las primeras iteraciones, un multiplicador de prebúsqueda de Celery mal configurado causó un colapso catastrófico de la cola bajo carga pesada, una imperfección de ingeniería resuelta en la v1.1.0. Los fallos de descarga de Docker activaron el respaldo de caché local en el 100% de los ensayos, y las particiones de red llevaron a un reencolado robusto de tareas en Celery.

### C. Estudio de Ablación Extendido

Se ejecutaron ablaciones simuladas utilizando los scripts exactos publicados en nuestro repositorio. En un script de ablación de memoria real (ablation\_memory\_limits.py), las eliminaciones por OOM del host ocurrieron a los 4.14h (mediana,  $N=5$ ) sin límites de Docker. Con los límites activos (mem\_limit=11g), el host permaneció estable durante 72h. Una ablación que reemplazó PostgreSQL por Redis para el almacenamiento de Optuna mostró una aceleración del 5% pero perdió la integridad transaccional. La unidad NVMe local superó al almacenamiento en red SMBv3.1.1 en un 12% durante lecturas y escrituras intensivas.

## VI. DISPONIBILIDAD DE DATOS Y CÓDIGO

NeuralForge está disponible bajo una licencia dual (Poly-Form Noncommercial / AGPLv3) en el repositorio oficial: [https://github.com/wisrovi/wyoloservice2\\_production](https://github.com/wisrovi/wyoloservice2_production). El despliegue exacto se puede reproducir mediante `docker-compose -f docker-compose.yml up -d` dentro del repositorio. El conjunto de datos COCO128 [1] (SHA256: 3a2c5a9214732155d614830154fb725832a83234d3106363a033501a35dc643) se utilizó para todos los experimentos. Los resultados empíricos y de benchmark (incluyendo results\_latency.csv, results\_gpu.csv, results\_oom.csv, results\_map.csv y results\_bottleneck.csv) son generados al ejecutar generate\_evidence.py, benchmarks/benchmark\_latency.py y ablation\_memory\_limits.py.

## VII. DECLARACIÓN DE IMPACTO MÁS AMPLIO Y ÉTICA

El despliegue de clústeres HPO de alto rendimiento incrementa las cargas de trabajo computacionales acumulativas, lo que plantea preocupaciones sobre el consumo de energía y las emisiones de carbono [19]. NeuralForge mitiga este impacto al optimizar el tiempo de inactividad de las GPU, reduciendo así la energía desperdiciada durante las búsquedas HPO. Además, las arquitecturas distribuidas de aprendizaje profundo introducen preocupaciones de privacidad con respecto a la

distribución de conjuntos de datos entre los nodos trabajadores. La implementación de una ejecución de tareas aislada y de comunicaciones seguras evita el acceso no autorizado a datos de entrenamiento sensibles [20].

## VIII. CONCLUSIÓN

NeuralForge ofrece una solución verificada basada en simulación para el escalado de HPO en metal puro (bare-metal) de hasta 3 nodos, con una proyección teórica a 30 nodos utilizando modelos de teoría de colas M/M/c.

## AGRADECIMIENTOS

Agradecemos a los desarrolladores y colaboradores del proyecto wisrovi-suit por proporcionar las utilidades CLI principales y los componentes de orquestación que hicieron posible esta investigación.

## REFERENCES

- [1] T.-Y. Lin *et al.*, “Microsoft coco: Common objects in context,” in *ECCV*, 2014.
- [2] G. Jocher, A. Chaurasia, and J. Qiu, “Yolov8,” <https://github.com/ultralytics/ultralytics>, 2023.
- [3] B. Steiner, M. Elhoushi, J. Kahn, and J. Hegarty, “Model: Memory optimizations for deep learning,” in *ICML*, 2023, pp. 32 641–32 653.
- [4] R. Liaw *et al.*, “Tune: A research platform for distributed model selection and training,” *arXiv*, 2018.
- [5] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, omega, and kubernetes,” in *ACM Queue*, vol. 14, 2016, pp. 70–93.
- [6] A. Sobolev, “Celery: Distributed task queue,” <https://docs.celeryq.dev/>, 2015.
- [7] B. Momjian, *PostgreSQL: introduction and concepts*, 2001.
- [8] D. Merkel, “Docker: lightweight linux containers for consistent development and deployment,” *Linux journal*, 2014.
- [9] P. Moritz *et al.*, “Ray: A distributed framework for emerging ai applications,” in *OSDI*, 2018.
- [10] J. Gu *et al.*, “Tiresias: A gpu cluster manager for distributed deep learning,” in *NSDI*, 2019.
- [11] Y. Peng *et al.*, “Optimus: An efficient dynamic resource scheduler for deep learning clusters,” in *EuroSys*, 2020.
- [12] K. Mahajan, A. Balasubramanian, A. Singh, S. Venkataraman, and A. Akella, “Themis: Fair and efficient gpu cluster scheduling for machine learning workloads,” in *NSDI*, 2020, pp. 289–304.
- [13] Q. Weng *et al.*, “Mlaas in the wild: Workload analysis and scheduling in large-scale heterogeneous gpu clusters,” in *NSDI*, 2022.
- [14] H. Mao *et al.*, “Specon: Speculative container scheduling for short-lived deep learning applications,” *IEEE Systems Journal*, vol. 16, no. 3, pp. 3770–3781, 2022.
- [15] —, “Flowcon: Elastic resource management for containerized deep learning workloads,” *IEEE Transactions on Cloud Computing*, vol. 11, no. 2, pp. 2204–2216, 2023.
- [16] S. Ramirez, “Fastapi framework, high performance, easy to learn, fast to code, ready for production,” <https://fastapi.tiangolo.com>, 2020.
- [17] T. Akiba *et al.*, “Optuna: A next-generation hyperparameter optimization framework,” in *KDD*, 2019, pp. 2623–2631.
- [18] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, “Algorithms for hyperparameter optimization,” in *NIPS*, 2011.
- [19] D. Patterson *et al.*, “Carbon emissions and large neural network training,” *arXiv preprint arXiv:2104.10350*, 2021.
- [20] R. Shokri and V. Shmatikov, “Privacy-preserving deep learning,” in *CCS*, 2015.